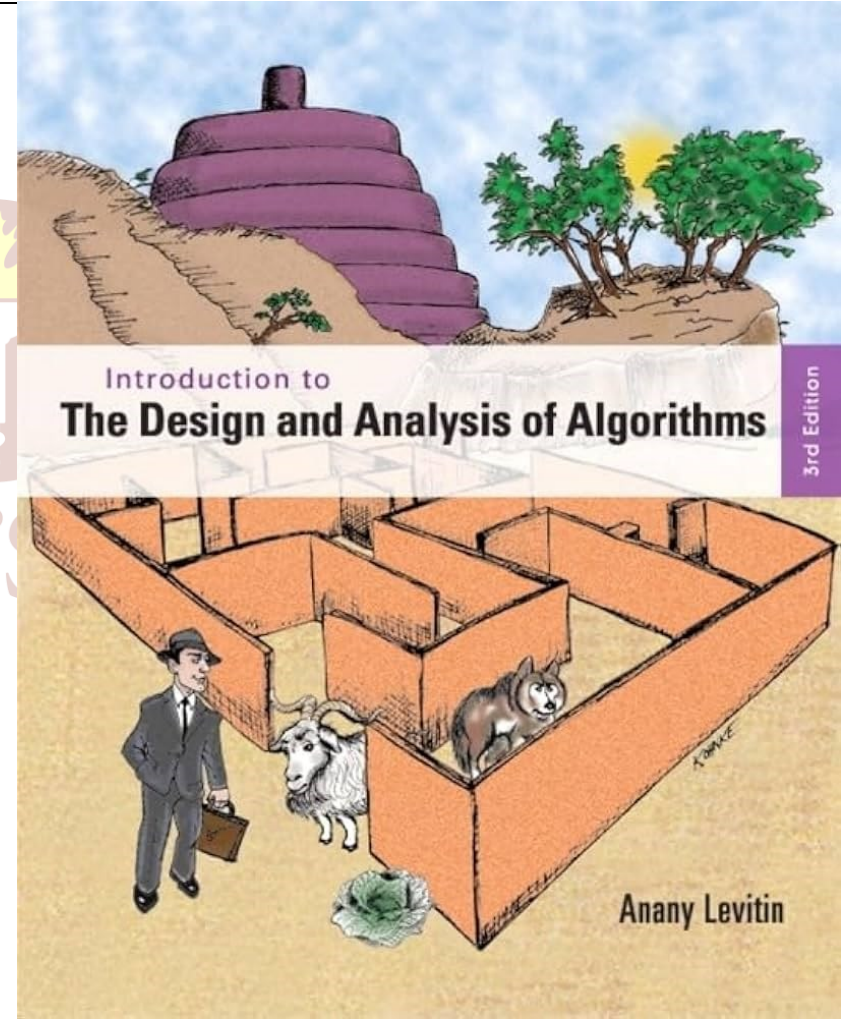


CS 07340 DESIGN AND ANALYSIS OF ALGORITHMS

WEEK 9 OVERVIEW

- Dynamic Programming
 - The Knapsack Problem and Memory Functions
 - Optimal Binary Search Tree



DYNAMIC PROGRAMMING

In [dynamic programming](#) an algorithmic problem is first broken down into sub-problems recursively. Then optimal solutions for the sub-problems are constructed and saved. The overall solution is created from the solutions to the subproblems.

An example of this technique is the calculation of Fibonacci numbers by the recurrence $F_n = F_{n-1} + F_{n-2}$, where $F_0 = 0$ and $F_1 = 1$. If we try to use this recurrence directly to compute the n -th Fibonacci number, we would have to recompute the same values of this function many times (since F_{n-1} and F_{n-2} both use all previous F_k). Saving results F_k for $k < n$ means each value is calculated only once.

BASIC EXAMPLES

BINOMIAL COEFFICIENTS

Binomial coefficients are the coefficients in the binomial theorem

$$(a + b)^n = C(n, 0)a^n b^0 + C(n, 1)a^{n-1}b^1 + \cdots C(n, n)a^0 b^n$$

We know that there is a direct formula to calculate $C(n, k)$ given by

$$C(n, k) = \frac{n!}{k! (n - k)!}$$

However, we also know the recurrence (Pascal's Triangle)

$$C(n, k) = C(n - 1, k) + C(n - 1, k - 1)$$

with initial values $C(n, 0) = 1$ and $C(n, n) = 1$ for $n \geq k \geq 0$.

We can calculate the binomial coefficient $C(n, k)$ by filling a table:

ALGORITHM *Binomial*(n, k)

//Computes $C(n, k)$ by the dynamic programming algorithm

//Input: A pair of nonnegative integers $n \geq k \geq 0$

//Output: The value of $C(n, k)$

for $i \leftarrow 0$ **to** n **do**

for $j \leftarrow 0$ **to** $\min(i, k)$ **do**

if $j = 0$ **or** $j = i$

$C[i, j] \leftarrow 1$

else $C[i, j] \leftarrow C[i - 1, j - 1] + C[i - 1, j]$

return $C[n, k]$

The runtime complexity is $\Theta(nk)$ and the memory requirements are $\Theta(nk)$.

COIN-ROW PROBLEM

There is a row of n coins whose values are some positive integers $c_1, c_2, \dots, c_n$, not necessarily distinct. The goal is to pick up the maximum amount of money subject to the constraint that no two coins adjacent in the initial row can be picked up.

The constraint can be viewed as saying we want all subsets that do not contain consecutive elements. For each coin we need to make the decision whether to include it or not. After n iterations, we have an answer using exhaustive search (similar to the construction of all 2^n subsets).

We will try devise a non-exhaustive search using recurrence. Let $F(n)$ be the maximum amount that can be picked up from the row of n coins. Divide all possible selections into those that do include the last coin c_n , and those which do not. Then

$$F(n) = \max\{c_n + F(n - 2), F(n - 1)\}$$

where the first number is the maximum when using c_n , and the second number $F(n - 1)$ is the maximum when excluding c_n . We have initial conditions

$$F(0) = 0 \text{ and } F(1) = c_1.$$

ALGORITHM *CoinRow*($C[1..n]$)

//Applies formula (8.3) bottom up to find the maximum amount of money

//that can be picked up from a coin row without picking two adjacent coins

//Input: Array $C[1..n]$ of positive integers indicating the coin values

//Output: The maximum amount of money that can be picked up

$F[0] \leftarrow 0$; $F[1] \leftarrow C[1]$

for $i \leftarrow 2$ **to** n **do**

$F[i] \leftarrow \max(C[i] + F[i - 2], F[i - 1])$

return $F[n]$



Example: Solve the coin-row problem by dynamic programming for the coin row 5, 1, 2, 10, 6, 2.

$$F[0] = 0, F[1] = c_1 = 5$$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5					

$$F[2] = \max\{1 + 0, 5\} = 5$$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5				

$$F[3] = \max\{2 + 5, 5\} = 7$$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7			

$$F[4] = \max\{10 + 5, 7\} = 15$$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15		

$$F[5] = \max\{6 + 7, 15\} = 15$$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15	15	

$$F[6] = \max\{2 + 15, 15\} = 17$$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15	15	17



CHANGE-MAKING PROBLEM

Give change for amount n using the minimum number of coins of denominations $d_1 < d_2 < \dots < d_m$. Note that these may not necessarily be ¢1, ¢5, ¢10, ¢25, ¢50, or \$1.



We consider a dynamic programming algorithm for the general case, assuming availability of unlimited quantities of coins for each of the m denominations where $d_1 = 1$.

Let $F(n)$ be the **minimum number of coins** whose values add up to n . We start with $F(0) = 0$. The amount n can only be obtained by adding one coin of denomination d_j to the amount $n - d_j$ for any $j = 1, 2, \dots$ where $n \geq d_j$. Therefore, we can consider all such denominations and select the one minimizing $F(n - d_j) + 1$. We have

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1$$

ALGORITHM *ChangeMaking*($D[1..m], n$)

//Applies dynamic programming to find the minimum number of coins

//of denominations $d_1 < d_2 < \dots < d_m$ where $d_1 = 1$ that add up to a

//given amount n

//Input: Positive integer n and array $D[1..m]$ of increasing positive

// integers indicating the coin denominations where $D[1] = 1$

//Output: The minimum number of coins that add up to n

$F[0] \leftarrow 0$

for $i \leftarrow 1$ **to** n **do**

$temp \leftarrow \infty$; $j \leftarrow 1$

while $j \leq m$ **and** $i \geq D[j]$ **do**

$temp \leftarrow \min(F[i - D[j]], temp)$

$j \leftarrow j + 1$

$F[i] \leftarrow temp + 1$

return $F[n]$



$$F[0] = 0$$

n	0	1	2	3	4	5	6
F	0						

$$F[1] = \min\{F[1 - 1]\} + 1 = 1$$

n	0	1	2	3	4	5	6
F	0	1					

$$F[2] = \min\{F[2 - 1]\} + 1 = 2$$

n	0	1	2	3	4	5	6
F	0	1	2				

$$F[3] = \min\{F[3 - 1], F[3 - 3]\} + 1 = 1$$

n	0	1	2	3	4	5	6
F	0	1	2	1			

$$F[4] = \min\{F[4 - 1], F[4 - 3], F[4 - 4]\} + 1 = 1$$

n	0	1	2	3	4	5	6
F	0	1	2	1	1		

$$F[5] = \min\{F[5 - 1], F[5 - 3], F[5 - 4]\} + 1 = 2$$

n	0	1	2	3	4	5	6
F	0	1	2	1	1	2	

$$F[6] = \min\{F[6 - 1], F[6 - 3], F[6 - 4]\} + 1 = 2$$

n	0	1	2	3	4	5	6
F	0	1	2	1	1	2	2

FIGURE 8.2 Application of Algorithm *MinCoinChange* to amount $n = 6$ and coin denominations 1, 3, and 4.

THE KNAPSACK PROBLEM AND MEMORY FUNCTIONS

To design a dynamic programming algorithm, we need to derive a recurrence relation that expresses a solution to an instance of the knapsack problem in terms of solutions to its smaller subinstances.

Let us consider an instance defined by the first i items, $1 \leq i \leq n$ with weights $w_1, \dots, w_i$ and values $v_1, \dots, v_i$, and suppose we have capacity $j < W$. Let $F(i, j)$ be the value of an optimal solution to this instance (so we have a two-dimensional table of partial solutions).

1. If item i is not used, then an optimal solution is $F(i - 1, j)$.
2. If item i is used, then an optimal solution is $F(i - 1, j - w_i) + v_i$.

$$F(i, j) = \begin{cases} \max\{F(i-1, j), F(i-1, j-w_i) + v_i\} & \text{if } j - w_i \geq 0 \\ F(i-1, j) & \text{if } j - w_i < 0 \end{cases}$$

with initial conditions

$$F(0, j) = 0 \quad \text{for } j \geq 0$$

$$F(i, 0) = 0 \quad \text{for } i \geq 0$$

Rowan University

	0	$j-w_i$	j	W
0	0	0	0	0
w_i, v_i	$i-1$	0	$F(i-1, j-w_i)$	$F(i-1, j)$
	i	0	$F(i, j)$	
n	0			goal

Example: Let us consider the instance given by the following data:

item	weight	value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

capacity $W = 5$

Use dynamic programming to find the maximum value.



$$w_1 = 2, v_1 = 12$$

$$w_2 = 1, v_2 = 10$$

$$w_3 = 3, v_3 = 20$$

$$w_4 = 2, v_4 = 15$$

i	capacity j					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

Rowan University

MEMORY FUNCTIONS

As we have seen, dynamic programming solves problems that have a recurrence relation. Using the recurrences directly in a recursive algorithm is a **top-down** technique (as with the Fibonacci numbers). It has the disadvantage that it solves common subproblems multiple times. The classic dynamic programming approach works **bottom-up** by filling a table with solutions to all smaller subproblems, each of them solved only once. Some of these smaller subproblems may not be necessary for getting a solution to the problem given, though. A more efficient way is to combine both approaches using memory functions: solve only subproblems that are necessary and solve these subproblems only once.

ALGORITHM *MFKnapsack*(i, j)

//Implements the memory function method for the knapsack problem
//Input: A nonnegative integer i indicating the number of the first
// items being considered and a nonnegative integer j indicating
// the knapsack capacity
//Output: The value of an optimal feasible subset of the first i items
//Note: Uses as global variables input arrays *Weights*[1.. n], *Values*[1.. n],
//and table $F[0..n, 0..W]$ whose entries are initialized with -1 's except for
//row 0 and column 0 initialized with 0's
if $F[i, j] < 0$
 if $j < \text{Weights}[i]$
 $\text{value} \leftarrow \text{MFKnapsack}(i - 1, j)$
 else
 $\text{value} \leftarrow \max(\text{MFKnapsack}(i - 1, j),$
 $\text{Values}[i] + \text{MFKnapsack}(i - 1, j - \text{Weights}[i]))$
 $F[i, j] \leftarrow \text{value}$
return $F[i, j]$

Example (continued):

		capacity j					
	i	0	1	2	3	4	5
	0	0	0	0	0	0	0
$w_1 = 2, v_1 = 12$	1	0	0	12	12	12	12
$w_2 = 1, v_2 = 10$	2	0	—	12	22	—	22
$w_3 = 3, v_3 = 20$	3	0	—	—	22	—	32
$w_4 = 2, v_4 = 15$	4	0	—	—	—	—	37

OPTIMAL BINARY SEARCH TREES

An optimal binary search tree (Optimal BST) is a binary search tree which provides the smallest possible search time for a given sequence of accesses. Optimal BSTs are weight-balanced binary trees. ***Weight balanced trees are height balanced*** (see Lemma).

In the ***static optimality problem***, the tree cannot be modified after it has been constructed. In the ***dynamic optimality problem***, the tree can be modified at any time, typically by permitting tree rotations.

LEMMA 3.1. *If T is an α -weight-balanced binary search tree, then T is α -height-balanced.*

PROBLEM DEFINITION

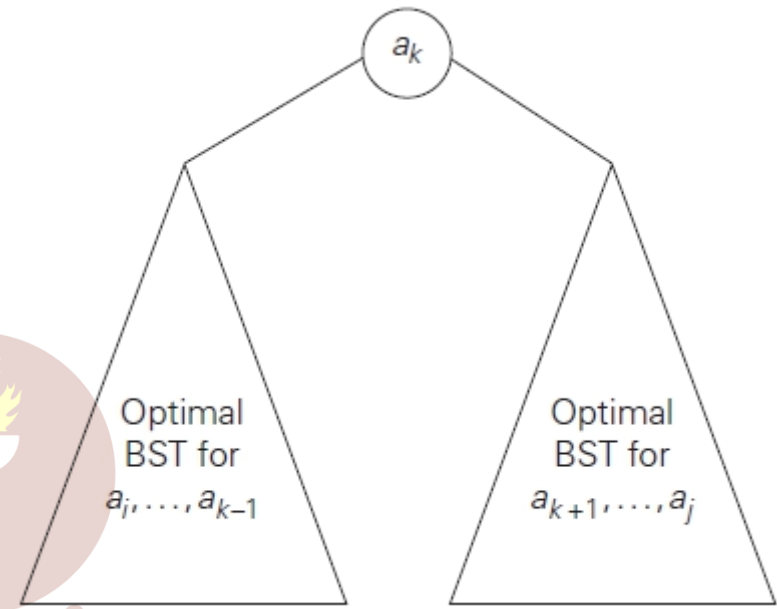
Let $a_1, a_2, \dots, a_n$ be distinct keys ordered from the smallest to the largest and let $p_1, p_2, \dots, p_n$ be the probabilities of searching for them. Let $C(i, j)$ be the smallest average number of comparisons made in a successful search in a binary search tree T_i^j for a range of keys $a_i, \dots, a_j$ ($1 \leq i \leq j \leq n$). Eventually, we are interested just in $C(1, n)$.

GENERAL DEFINITION

Note that we know the solution for a uniform distribution with $p_i = \frac{1}{n}$. In that case, we should use medians as roots. In general, the probabilities will skew the 50% - 50% weight-balance, though.

There is a [more general problem definition due to Donald Knuth](#). In it, we have n probabilities for successful search (like in our definition), but in addition there are $n + 1$ probabilities for failed searches (together these $2n + 1$ probabilities cover the search space for all keys, not just the given n keys). The number of possible trees for a given set of n keys is $C_n = \binom{2n}{n} \frac{1}{n+1}$ (known as [Catalan numbers](#)). They grow exponentially since $C_n \approx \frac{4^n}{n^{3/2}\sqrt{\pi}}$. Hence exhaustive search is infeasible for large n .

We will consider all possible ways to choose a root a_k among the keys. The left subtree T_i^{k-1} must contain keys $a_i, \dots, a_{k-1}$ optimally arranged, and the right subtree T_{k+1}^j contains keys $a_{k+1}, \dots, a_j$ also optimally arranged.



$$C(i, j) = \min_{i \leq k \leq j} \{C(i, k-1) + C(k+1, j)\} + \sum_{s=i}^j p_s$$

	0	1					j	n
1	0	p_1						goal
		0	p_2					
i							$C[i,j]$	
								p_n
								0
$n+1$								



Rowan
University

ALGORITHM *OptimalBST*($P[1..n]$)

//Finds an optimal binary search tree by dynamic programming

//Input: An array $P[1..n]$ of search probabilities for a sorted list of n keys

//Output: Average number of comparisons in successful searches in the

// optimal BST and table R of subtrees' roots in the optimal BST

for $i \leftarrow 1$ **to** n **do**

$C[i, i - 1] \leftarrow 0$

$C[i, i] \leftarrow P[i]$

$R[i, i] \leftarrow i$

$C[n + 1, n] \leftarrow 0$

for $d \leftarrow 1$ **to** $n - 1$ **do** //diagonal count

for $i \leftarrow 1$ **to** $n - d$ **do**

$j \leftarrow i + d$

$minval \leftarrow \infty$

for $k \leftarrow i$ **to** j **do**

if $C[i, k - 1] + C[k + 1, j] < minval$

$minval \leftarrow C[i, k - 1] + C[k + 1, j]; \quad kmin \leftarrow k$

$R[i, j] \leftarrow kmin$

$sum \leftarrow P[i]; \quad \textbf{for } s \leftarrow i + 1 \textbf{ to } j \textbf{ do } sum \leftarrow sum + P[s]$

$C[i, j] \leftarrow minval + sum$

return $C[1, n], R$



OPTIMAL BST RUNTIME ANALYSIS

The dynamic programming algorithm by Gilbert and Moore introduced in 1958 required $O(n^3)$ time and $O(n^2)$ space. Knuth improved their algorithm with heuristics to implement **nearly** optimal binary search trees in $O(n \log(n))$ time and $O(n)$ space.



QUIZ TOPICS

- Memory function and Optimal BST terminology

